

DEPARTMENT OF INFORMATION TECHNOLOGY

24ITT32 - PROGRAMMING IN JAVA

HIGH-FREQUENCY ALGORITHM STOCK TRADING &ORDER EXECUTION ENGINE

P R O J E C T R E P O R T

Submitted by

THULAI S

25ITR163

SUBASREE G

25ITR144

JAVA ASSIGNMENT

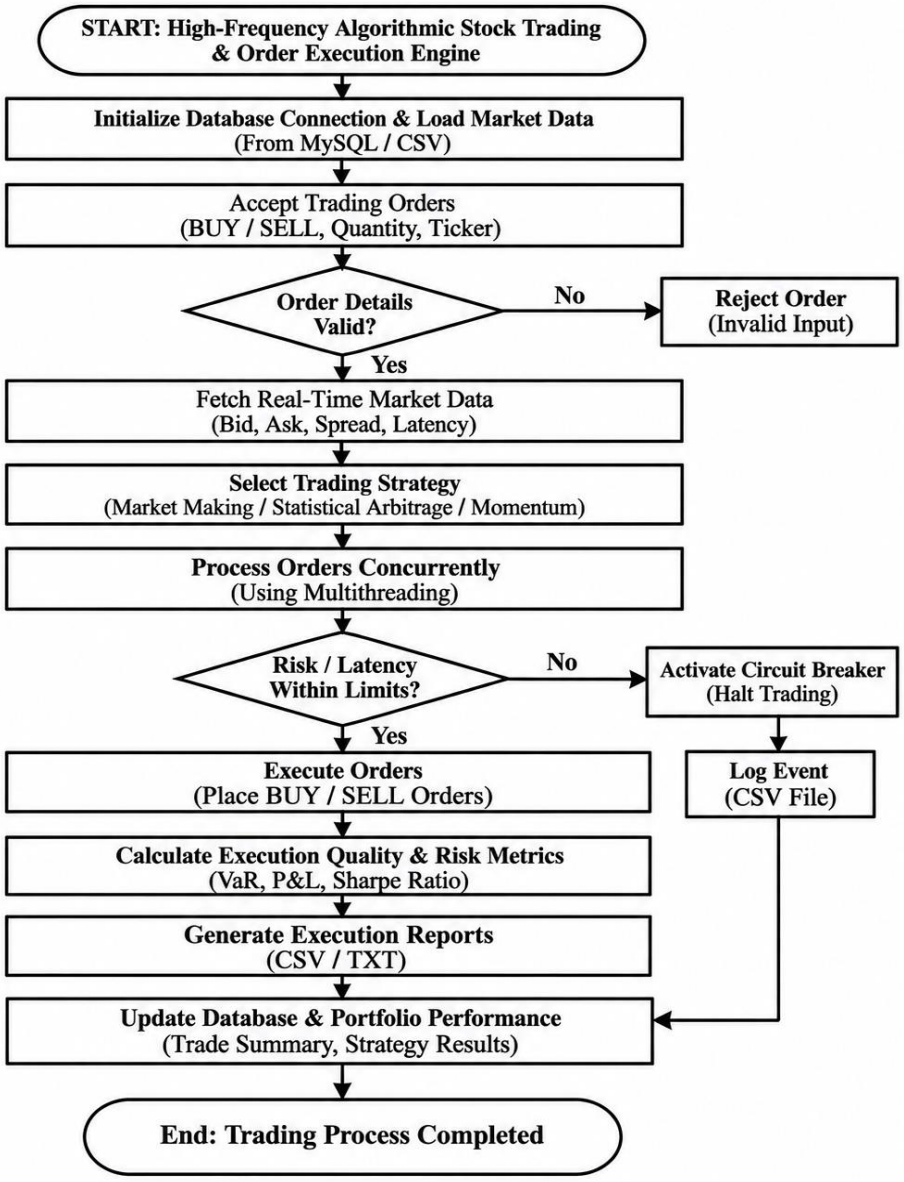
AIM:

To design and develop a High-Frequency Algorithmic Stock Trading and Order Execution Engine using Java and JDBC to simulate automated stock order processing, evaluate algorithmic trading strategies, monitor market conditions, calculate portfolio risk, execute multiple orders concurrently, and generate execution reports using a MySQL database.

OBJECTIVE:

The objective of this project is to develop a Java-based algorithmic trading system that efficiently processes BUY and SELL orders and evaluates them using different trading strategies such as Market Making, Statistical Arbitrage and High-Volatility Momentum

FLOW CHART:



PROCEDURE :

- Create a Java project in VS Code and organize the required source files.
- Create the MySQL database and execute the SQL scripts to create the required tables.
- Add the MySQL Connector/J library and configure database details in DBConnection.java.
- Establish database connectivity using Java JDBC.
- Run MainApp.java to start the trading application.
- Enter BUY/SELL order details such as Order ID, Trader ID, Stock Ticker, Price and Quantity.
- Validate the order details and store valid orders in the MySQL database using TradingDAO.java.
- Record market information such as bid price, ask price, spread and network latency using MarketTelemetry.java.
- Select a trading strategy such as Market Making, Statistical Arbitrage or High-Volatility Momentum.
- Evaluate trading orders using StrategyEvaluator.java and process multiple orders using ExecutionWorker.java and multithreading.
- Calculate portfolio risk using RiskManager.java and check the risk and latency limits.
- If limits are exceeded, activate the circuit breaker and record the risk event; otherwise, execute the simulated BUY/SELL orders.
- Calculate execution quality and update the execution details in the database.
- Generate execution reports and circuit breaker logs in CSV format using FillReportWriter.java.
- Verify the database records and generated reports to complete the trading process.

DATABASE SCHEMA:

1.ORDERS TABLE:

```
mysql> SELECT * FROM orders;
+------------------+------------+--------+------+-------------+----------+-----------+--------------------------+--------------------------+
| order_id         | trader_id  | ticker | side | limit_price | quantity | status    | created_at               | updated_at               |
+------------------+------------+--------+------+-------------+----------+-----------+--------------------------+--------------------------+
| ORD1001          | Suba       | TCS    | BUY  | 5000.000000 |        6 | CANCELLED | 2026-11-09 21:11:42.519808 | 2026-11-09 21:19:32.815145 |
| SAMPLE-EXEC-01   | demo-trader | AAPL   | BUY  |  190.020000 |       10 | EXECUTED  | 2026-11-09 20:52:04.608732 | 2026-11-09 20:52:04.608732 |
| SAMPLE-PENDING-01 | demo-trader | MSFT   | SELL |  410.000000 |        5 | PENDING   | 2026-11-09 20:52:04.608732 | 2026-11-09 20:52:04.608732 |
+------------------+------------+--------+------+-------------+----------+-----------+--------------------------+--------------------------+
3 rows in set (0.06 sec)
```

2.EXECUTIONS TABLE:

```
mysql> SELECT * FROM executions;
+--------------+----------------+--------------+----------------+------------+---------------+-----------------+----------------------------+
| execution_id | order_id       | filled_price | slippage_ticks | commission | quality_score | quality_grade   | executed_at                |
+--------------+----------------+--------------+----------------+------------+---------------+-----------------+----------------------------+
|            1 | SAMPLE-EXEC-01 |   190.020000 |              1 |   1.900200 |         91.20 | Alpha Execution | 2025-01-15 12:00:00.123456 |
+--------------+----------------+--------------+----------------+------------+---------------+-----------------+----------------------------+
1 row in set (0.01 sec)
```

3.ORDER BACKLOGS TABLE:

```
mysql> SELECT * FROM order_book_logs;
+--------+--------+------------+------------+----------+------------+-----------------+--------------------+--------------+-----------------+----------------------------+
| log_id | ticker | bid_price  | ask_price  | spread   | latency_ms | market_depth_lots | price_divergence_bps | momentum_bps | inventory_units | recorded_at                |
+--------+--------+------------+------------+----------+------------+-----------------+--------------------+--------------+-----------------+----------------------------+
|      1 | AAPL   | 190.000000 | 190.020000 | 0.020000 |   1.200000 |              25 |           2.500000 |     1.500000 |              20 | 2026-11-09 20:52:04.643102 |
|      2 | TCS    | 4490.000000 | 4500.000000 | 10.000000 |   2.000000 |             100 |           2.000000 |     1.000000 |              30 | 2026-11-09 21:17:35.054496 |
+--------+--------+------------+------------+----------+------------+-----------------+--------------------+--------------+-----------------+----------------------------+
2 rows in set (0.01 sec)
```

4.RISK BREACHES TABLES:

```
mysql> SELECT * FROM risk_breaches;
+-----------+-------------+--------------------+-----------------------------------------+-----------------+----------------------------+
| breach_id | var_percent | execution_latency_ms | reason                                  | action_taken    | recorded_at                |
+-----------+-------------+--------------------+-----------------------------------------+-----------------+----------------------------+
|         1 |    6.000000 |           1.200000 | SAMPLE ONLY: fictional VaR threshold exceeded | CIRCUIT_BREAKER_HALT | 2026-11-09 20:52:04.661140 |
+-----------+-------------+--------------------+-----------------------------------------+-----------------+----------------------------+
1 row in set (0.00 sec)
```

SOURCE CODE:

MAINAPP.JAVA:

```
import java.io.BufferedReader;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Scanner;
import java.util.UUID;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class MainApp {
    private static final Path ORDER_STREAM = Path.of("input", "order_stream.txt");
    private final Scanner scanner = new Scanner(System.in);
    private final TradingDAO dao = new TradingDAO();
    private final FillReportWriter reportWriter = new FillReportWriter();
    private final RiskManager riskManager = new RiskManager(dao, reportWriter);
    private StrategyEvaluator.Strategy selectedStrategy =
            StrategyEvaluator.Strategy.MARKET_MAKING;
    private double potentialLoss;
```

```
private double portfolioEquity = 100_000.0;
public static void main(String[] args) {
    new MainApp().run();
}
private void run() {
    demonstrateFixParsing();
    boolean running = true;
    while (running) {
        printMenu();
        String choice = prompt("Select an option: ");
        try {
            switch (choice) {
                case "1": addNewOrder(); break;
                case "2": viewAllOrders(); break;
                case "3": enterMarketTelemetry(); break;
                case "4": executeOrders(); break;
                case "5": selectStrategy(); break;
                case "6": calculateExecutionQuality(); break;
                case "7": checkPortfolioVaR(); break;
                case "8": viewCircuitBreakerStatus(); break;
                case "9": viewExecutionHistory(); break;
                case "10": generateFillReport(); break;
                case "11": readOrderStreamFile(); break;
                case "12":
                    running = false;
                    System.out.println("Trading engine stopped.");
                    break;
```

```
                default:
                    System.out.println("Enter a menu option from 1 to 12.");
            }
        } catch (IllegalArgumentException exception) {
            System.err.println("Invalid input: " + exception.getMessage());
        } catch (SQLException | IOException exception) {
            System.err.println("Operation failed: " + exception.getMessage());
        }
    }
}
private void printMenu() {
    System.out.println();
    System.out.println("========== HIGH FREQUENCY TRADING ENGINE ==========");
    System.out.println("1. Add New Order");
    System.out.println("2. View All Orders");
    System.out.println("3. Enter Market Telemetry");
    System.out.println("4. Execute Orders");
    System.out.println("5. Select Trading Strategy");
    System.out.println("6. Calculate Execution Quality Score");
    System.out.println("7. Check Portfolio VaR");
    System.out.println("8. View Circuit Breaker Status");
    System.out.println("9. View Execution History");
    System.out.println("10. Generate Fill Report");
    System.out.println("11. Read Order Stream File");
    System.out.println("12. Exit");
    System.out.println("================================================");
```

```
    }
    private void addNewOrder() throws SQLException {
        if (riskManager.isHalted()) {
            System.out.println("Order rejected: the circuit breaker is active.");
            return;
        }
        String orderId = prompt("Order ID (leave blank to generate one): ").trim();
        if (orderId.isEmpty()) {
            orderId = "ORD-" + UUID.randomUUID().toString().substring(0, 8);
        }
        String traderId = prompt("Trader ID: ").trim();
        String ticker = prompt("Ticker (A-Z, 1-5 letters): ").trim();
        Order.Side side = Order.Side.valueOf(prompt("Side (BUY/SELL):
").trim().toUpperCase());
        double limitPrice = readDouble("Limit price: ");
        int quantity = readInt("Quantity: ");
        Order order = new Order(orderId, traderId, ticker, side, limitPrice, quantity);
        dao.addOrder(order);
        System.out.println("Order accepted and saved as PENDING: " + order.getOrderId());
    }
    private void viewAllOrders() throws SQLException {
        printRows("Orders", dao.findAllOrders());
    }
    private void enterMarketTelemetry() throws SQLException {
        MarketTelemetry telemetry = promptTelemetry();
        dao.saveTelemetry(telemetry);
        System.out.println("Telemetry saved for " + telemetry.getTicker()
```

```
            + " (spread " + String.format("%.4f", telemetry.getSpread()) + ").");
    }
    private MarketTelemetry promptTelemetry() {
        String ticker = prompt("Ticker: ").trim();
        double bid = readDouble("Bid price: ");
        double ask = readDouble("Ask price: ");
        double latency = readDouble("Network latency (ms): ");
        int depth = readInt("Market depth (lots): ");
        double divergence = readDouble("Price divergence (basis points, e.g. 2.5): ");
        double momentum = readDouble("Momentum vector (basis points, e.g. 1.5): ");
        int inventory = readInt("Current inventory units: ");
        return new MarketTelemetry(ticker, bid, ask, latency, depth, divergence, momentum,
inventory);
    }
    private void executeOrders() throws SQLException, IOException {
        if (riskManager.isHalted()) {
            System.out.println("Execution blocked: circuit breaker is active.");
            return;
        }
        List<Order> orders = dao.findPendingOrders();
        if (orders.isEmpty()) {
            System.out.println("There are no pending orders.");
            return;
        }
        int threadCount = Math.max(2, Math.min(8,
Runtime.getRuntime().availableProcessors()));
        ExecutorService executor = Executors.newFixedThreadPool(threadCount);
        try {
```

```
    List<ExecutionWorker> workers = new ArrayList<>();
    for (Order order : orders) {
        ExecutionWorker worker = new ExecutionWorker(
                order, selectedStrategy, dao, riskManager, reportWriter,
                potentialLoss, portfolioEquity);
        workers.add(worker);
        executor.execute(worker);
    }
    for (ExecutionWorker worker : workers) {
        try {
            System.out.println(worker.getResult());
        } catch (InterruptedException exception) {
            Thread.currentThread().interrupt();
            throw new IOException("Order execution was interrupted.", exception);
        } catch (ExecutionException exception) {
            Throwable cause = exception.getCause();
            if (cause instanceof SQLException) {
                throw (SQLException) cause;
            }
            if (cause instanceof IOException) {
                throw (IOException) cause;
            }
            throw new IOException("An order worker failed.", cause);
        }
    }
} finally {
    executor.shutdown();
```

```
        }
    }

    private void selectStrategy() {
        System.out.println("1. MARKET MAKING (maximum latency 5 ms)");
        System.out.println("2. STATISTICAL ARBITRAGE (maximum latency 2 ms)");
        System.out.println("3. HIGH-VOLATILITY MOMENTUM (maximum latency 10
ms)");
        switch (prompt("Select strategy: ").trim()) {
            case "1":
                selectedStrategy = StrategyEvaluator.Strategy.MARKET_MAKING;
                break;
            case "2":
                selectedStrategy = StrategyEvaluator.Strategy.STATISTICAL_ARBITRAGE;
                break;
            case "3":
                selectedStrategy =
StrategyEvaluator.Strategy.HIGH_VOLATILITY_MOMENTUM;
                break;
            default:
                throw new IllegalArgumentException("Select strategy 1, 2 or 3.");
        }
        System.out.println("Selected strategy: " + selectedStrategy);
    }

    private void calculateExecutionQuality() {
        double latency = readDouble("Measured latency (ms): ");
        long slippageTicks = readLong("Execution slippage (ticks): ");
```

```
        double spread = readDouble("Bid/ask spread: ");
        double ask = readDouble("Ask price: ");
        StrategyEvaluator.Evaluation evaluation = StrategyEvaluator.executionQuality(
                latency, slippageTicks, spread, ask, riskManager.isHalted());
        System.out.printf("Execution quality: %.2f / 100 — %s%n",
                evaluation.getScore(), evaluation.getGrade());
    }

    private void checkPortfolioVaR() throws SQLException, IOException {
        potentialLoss = readDouble("Potential portfolio loss: ");
        portfolioEquity = readDouble("Total portfolio equity: ");
        double varPercent = riskManager.calculateVaRPercent(potentialLoss, portfolioEquity);
        boolean allowed = riskManager.assess(potentialLoss, portfolioEquity, 0,
                selectedStrategy.getMaximumLatencyMs());
        System.out.printf("VaR = %.2f%% (limit 5.00%%).%n", varPercent);
        if (!allowed) {
            System.out.println("Account halted: pending orders were cancelled and the breach
was logged.");
        } else {
            System.out.println("VaR check passed.");
        }
    }

    private void viewCircuitBreakerStatus() {
        System.out.println("Circuit breaker: "
                + (riskManager.isHalted() ? "CIRCUIT_BREAKER_HALT (active)" : "inactive"));
    }
```

```
    private void viewExecutionHistory() throws SQLException {
        printRows("Execution history", dao.findExecutionHistory());
    }

    private void generateFillReport() throws SQLException, IOException {
        List<String> rows = dao.findAllExecutionCsvRows();
        reportWriter.writeAllFills(rows);
        System.out.println("Wrote " + rows.size() + " execution row(s) to "
                + "output/execution_fills.csv.");
    }

    private void readOrderStreamFile() throws IOException, SQLException {
        if (riskManager.isHalted()) {
            System.out.println("Order stream not imported: the circuit breaker is active.");
            return;
        }
        if (!Files.isRegularFile(ORDER_STREAM)) {
            throw new IOException("Input file not found: " +
ORDER_STREAM.toAbsolutePath());
        }
        int accepted = 0;
        int rejected = 0;
        try (BufferedReader reader = Files.newBufferedReader(ORDER_STREAM,
StandardCharsets.UTF_8)) {
            String line;
            int lineNumber = 0;
            while ((line = reader.readLine()) != null) {
```

```
lineNumber++;
if (line.isBlank() || line.trim().startsWith("#")) {
    continue;
}
try {
    String[] fields = line.split(",", -1);
    if (fields.length != 13) {
        throw new IllegalArgumentException(
                "Expected 13 comma-separated fields, found " + fields.length);
    }
    Order order = new Order(fields[0].trim(), fields[1].trim(), fields[2].trim(),
            Order.Side.valueOf(fields[3].trim().toUpperCase()),
            Double.parseDouble(fields[4].trim()),
            Integer.parseInt(fields[5].trim()));
    MarketTelemetry telemetry = new MarketTelemetry(fields[2].trim(),
            Double.parseDouble(fields[6].trim()),
            Double.parseDouble(fields[7].trim()),
            Double.parseDouble(fields[8].trim()),
            Integer.parseInt(fields[9].trim()),
            Double.parseDouble(fields[10].trim()),
            Double.parseDouble(fields[11].trim()),
            Integer.parseInt(fields[12].trim()));
    dao.addOrder(order);
    dao.saveTelemetry(telemetry);
    accepted++;
} catch (IllegalArgumentException | SQLException exception) {
    rejected++;
```

```
                System.err.println("Line " + lineNumber + " rejected: " +
exception.getMessage());
            }
        }
    }
    System.out.printf("Order stream complete: %d accepted, %d rejected.%n", accepted,
rejected);
  }

  private void demonstrateFixParsing() {
    String sample = "35=D|55=AAPL|54=1|38=100";
    Map<String, String> tags = parseFixMessage(sample);
    System.out.println("FIX sample parsed: 55=" + tags.get("55")
        + ", 54=" + tags.get("54") + " (35=D|55=AAPL|54=1|38=100)");
  }

  static Map<String, String> parseFixMessage(String message) {
    Map<String, String> tags = new HashMap<>();
    for (String field : message.split("\\|")) {
      int equalsIndex = field.indexOf('=');
      if (equalsIndex <= 0 || equalsIndex == field.length() - 1) {
        throw new IllegalArgumentException("Malformed FIX tag: " + field);
      }
      String[] parts = field.split("=");
      if (parts.length != 2 || parts[0].trim().isEmpty()) {
        throw new IllegalArgumentException("Malformed FIX tag: " + field);
      }
      tags.put(parts[0], parts[1]);
```

```
        }
        return tags;
    }
    private String prompt(String label) {
        System.out.print(label);
        return scanner.nextLine();
    }
    private double readDouble(String label) {
        try {
            return Double.parseDouble(prompt(label).trim());
        } catch (NumberFormatException exception) {
            throw new IllegalArgumentException("Enter a valid number for " + label.trim());
        }
    }
    private int readInt(String label)
try {
            return Integer.parseInt(prompt(label).trim());
        } catch (NumberFormatException exception) {
            throw new IllegalArgumentException("Enter a whole number for " + label.trim());
        }
    }
    private long readLong(String label) {
        try {
            return Long.parseLong(prompt(label).trim());
        } catch (NumberFormatException exception) {
            throw new IllegalArgumentException("Enter a whole number for " + label.trim());
        }
```

```
    }
    private void printRows(String heading, List<String> rows) {
        System.out.println("--- " + heading + " ---");
        if (rows.isEmpty()) {
            System.out.println("No records found.");
            return;
        }
        for (String row : rows) {
            System.out.println(row);
        }
    }
}
```

DB CONNECTION.JAVA:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
public final class DBConnection {
   private static final String URL =
"jdbc:mysql://localhost:3306/trading_engine";
    private static final String USER = "root";
    private static final String PASSWORD = "Suba@1324";
    private DBConnection() { }
    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

OUTPUT SCREENSHOTS:

MAIN MENU:

```
========== HIGH FREQUENCY TRADING ENGINE ==========
1. Add New Order
2. View All Orders
3. Enter Market Telemetry
4. Execute Orders
5. Select Trading Strategy
6. Calculate Execution Quality Score
7. Check Portfolio VaR
8. View Circuit Breaker Status
9. View Execution History
10. Generate Fill Report
11. Read Order Stream File
12. Exit
==================================================
```

1.ADD NEW ORDER :

```
========== HIGH FREQUENCY TRADING ENGINE ==========
1. Add New Order
2. View All Orders
3. Enter Market Telemetry
4. Execute Orders
5. Select Trading Strategy
6. Calculate Execution Quality Score
7. Check Portfolio VaR
8. View Circuit Breaker Status
9. View Execution History
10. Generate Fill Report
11. Read Order Stream File
12. Exit
==================================================
Select an option: 1
Order ID (leave blank to generate one): ORD1002
Trader ID: TRADER001
Ticker (A-Z, 1-5 letters): MSFT
Side (BUY/SELL): BUY
Limit price: 450
Quantity: 6
Order accepted and saved as PENDING: ORD1002
```

2.VIEW ALL ORDERS :

```
========== HIGH FREQUENCY TRADING ENGINE ==========
1. Add New Order
2. View All Orders
3. Enter Market Telemetry
4. Execute Orders
5. Select Trading Strategy
6. Calculate Execution Quality Score
7. Check Portfolio VaR
8. View Circuit Breaker Status
9. View Execution History
10. Generate Fill Report
11. Read Order Stream File
12. Exit
================================================
Select an option: 2
--- Orders ---
ORD1002 | TRADER001 | BUY MSFT | 450.0000 x 6 | PENDING | 2026-11-09 21:53:53.239624
ORD1001 | Suba | BUY TCS | 5000.0000 x 6 | CANCELLED | 2026-11-09 21:11:42.519808
SAMPLE-EXEC-01 | demo-trader | BUY AAPL | 190.0200 x 10 | EXECUTED | 2026-11-09 20:52:04.608732
SAMPLE-PENDING-01 | demo-trader | SELL MSFT | 410.0000 x 5 | PENDING | 2026-11-09 20:52:04.608732
```

3.ENTER MARKET TELEMETRY :

```
========== HIGH FREQUENCY TRADING ENGINE ==========
1. Add New Order
2. View All Orders
3. Enter Market Telemetry
4. Execute Orders
5. Select Trading Strategy
6. Calculate Execution Quality Score
7. Check Portfolio VaR
8. View Circuit Breaker Status
9. View Execution History
10. Generate Fill Report
11. Read Order Stream File
12. Exit
================================================
Select an option: 3
Ticker: MSFT
Bid price: 449.50
Ask price: 450.00
Network latency (ms): 25
Market depth (lots): 100
Price divergence (basis points, e.g. 2.5): 2
Momentum vector (basis points, e.g. 1.5): 1
Current inventory units: 1000
Telemetry saved for MSFT (spread 0.5000).
```

4.EXECUTE ORDERS :

```
========== HIGH FREQUENCY TRADING ENGINE ==========
1. Add New Order
2. View All Orders
3. Enter Market Telemetry
4. Execute Orders
5. Select Trading Strategy
6. Calculate Execution Quality Score
7. Check Portfolio VaR
8. View Circuit Breaker Status
9. View Execution History
10. Generate Fill Report
11. Read Order Stream File
12. Exit
================================================
Select an option: 4
CIRCUIT_BREAKER_HALT: Latency 25.000 ms exceeds 5.000 ms. Cancelled pending or
ders: 2
SAMPLE-PENDING-01 rejected: risk check failed.
ORD1002 rejected: risk check failed.
```

5.SELECT TRADING STRATEGY :

```
========== HIGH FREQUENCY TRADING ENGINE ==========
1. Add New Order
2. View All Orders
3. Enter Market Telemetry
4. Execute Orders
5. Select Trading Strategy
6. Calculate Execution Quality Score
7. Check Portfolio VaR
8. View Circuit Breaker Status
9. View Execution History
10. Generate Fill Report
11. Read Order Stream File
12. Exit
================================================
Select an option: 5
1. MARKET MAKING (maximum latency 5 ms)
2. STATISTICAL ARBITRAGE (maximum latency 2 ms)
3. HIGH-VOLATILITY MOMENTUM (maximum latency 10 ms)
Select strategy: 3
Selected strategy: HIGH_VOLATILITY_MOMENTUM
```

6.EXECUTION QUALITY SCORE :

```
========== HIGH FREQUENCY TRADING ENGINE ==========
1. Add New Order
2. View All Orders
3. Enter Market Telemetry
4. Execute Orders
5. Select Trading Strategy
6. Calculate Execution Quality Score
7. Check Portfolio VaR
8. View Circuit Breaker Status
9. View Execution History
10. Generate Fill Report
11. Read Order Stream File
12. Exit
================================================
Select an option: 6
Measured latency (ms): 25
Execution slippage (ticks): 2
Bid/ask spread: 0.50
Ask price: 450.00
Execution quality: 0.00 / 100 ? Trade Rejected
```

7.CHECK PORTFOLIO VaR:

```
========== HIGH FREQUENCY TRADING ENGINE ==========
1. Add New Order
2. View All Orders
3. Enter Market Telemetry
4. Execute Orders
5. Select Trading Strategy
6. Calculate Execution Quality Score
7. Check Portfolio VaR
8. View Circuit Breaker Status
9. View Execution History
10. Generate Fill Report
11. Read Order Stream File
12. Exit
================================================
Select an option: 7
Potential portfolio loss: 1000
Total portfolio equity: 100000
VaR = 1.00% (limit 5.00%).
Account halted: pending orders were cancelled and the breach was logged.
```

8.CIRCUIT BREAKER STATUS:

```
========== HIGH FREQUENCY TRADING ENGINE ==========
1. Add New Order
2. View All Orders
3. Enter Market Telemetry
4. Execute Orders
5. Select Trading Strategy
6. Calculate Execution Quality Score
7. Check Portfolio VaR
8. View Circuit Breaker Status
9. View Execution History
10. Generate Fill Report
11. Read Order Stream File
12. Exit
================================================
Select an option: 8
Circuit breaker: CIRCUIT_BREAKER_HALT (active)
```

9.EXECUTION HISTORY:

```
========== HIGH FREQUENCY TRADING ENGINE ==========
1. Add New Order
2. View All Orders
3. Enter Market Telemetry
4. Execute Orders
5. Select Trading Strategy
6. Calculate Execution Quality Score
7. Check Portfolio VaR
8. View Circuit Breaker Status
9. View Execution History
10. Generate Fill Report
11. Read Order Stream File
12. Exit
================================================
Select an option: 9
--- Execution history ---
SAMPLE-EXEC-01 | BUY AAPL | fill 190.0200 | 1 ticks | commission 1.9002 | score 91.2 (Alpha Execution) | 2025-01-15 12:00:00.123456
```

10.GENERATE FILL REPORT:

```
========== HIGH FREQUENCY TRADING ENGINE ==========
1. Add New Order
2. View All Orders
3. Enter Market Telemetry
4. Execute Orders
5. Select Trading Strategy
6. Calculate Execution Quality Score
7. Check Portfolio VaR
8. View Circuit Breaker Status
9. View Execution History
10. Generate Fill Report
11. Read Order Stream File
12. Exit
================================================
Select an option: 10
Wrote 1 execution row(s) to output/execution_fills.csv.
```

11.READ ORDER STREAM FILE:

```
========== HIGH FREQUENCY TRADING ENGINE ==========
1. Add New Order
2. View All Orders
3. Enter Market Telemetry
4. Execute Orders
5. Select Trading Strategy
6. Calculate Execution Quality Score
7. Check Portfolio VaR
8. View Circuit Breaker Status
9. View Execution History
10. Generate Fill Report
11. Read Order Stream File
12. Exit
====================================================
Select an option: 11
Order stream not imported: the circuit breaker is active.
```

12.EXIT:

```
========== HIGH FREQUENCY TRADING ENGINE ==========
1. Add New Order
2. View All Orders
3. Enter Market Telemetry
4. Execute Orders
5. Select Trading Strategy
6. Calculate Execution Quality Score
7. Check Portfolio VaR
8. View Circuit Breaker Status
9. View Execution History
10. Generate Fill Report
11. Read Order Stream File
12. Exit
====================================================
Select an option: 12
Trading engine stopped.
```

RESULT:

Thus, the High-Frequency Algorithmic Stock Trading and Order Execution Engine is developed successfully using Java, JDBC, MySQL and Multithreading to perform automated trading order processing, strategy evaluation, risk management and report generation.

REFERENCES:

1. Oracle, Java SE Documentation – JDBC Database Access and JDBC Basics.
https://docs.oracle.com/javase/tutorial/jdbc/

2. Oracle, Java SE API Documentation – Java Programming and Database Connectivity.
https://docs.oracle.com/en/java/javase/26/docs/api/

3. Oracle, Java SE API – PreparedStatement Interface.
https://docs.oracle.com/javase/8/docs/api/java/sql/PreparedStatement.html

4. Oracle, Java SE API – File Handling and BufferedReader.
https://docs.oracle.com/en/java/javase/26/docs/api/java.base/java/io/BufferedReader.html

5. Oracle, MySQL 8.0 Reference Manual – Database Management and SQL Operations.
https://dev.mysql.com/doc/refman/8.0/en/

6. Oracle, MySQL Connector/J Developer Guide – JDBC Connectivity with MySQL.
https://dev.mysql.com/doc/connector-j/en/

7. Oracle, Java SE Documentation – JDBC Prepared Statements and Database Transactions.
https://docs.oracle.com/javase/tutorial/jdbc/basics/prepared.html

8. Reserve Bank of India, Guidance Note on Market Risk Management.
https://www.rbi.org.in/Scripts/bs_viewcontent.aspx?Id=4788

9. Securities and Exchange Board of India (SEBI), Key Risks in Investing in Securities Market.
https://investor.sebi.gov.in/securities-risks_trade_derivatives.html

10. GeeksforGeeks, Establishing JDBC Connection in Java.
https://www.geeksforgeeks.org/establishing-jdbc-connection-in-java/

11. YouTube, Java JDBC with MySQL Tutorial Videos.
https://www.youtube.com/results?search_query=Java+JDBC+MySQL+tutorial

GITHUB:

LINK :	https://github.com/Thulasi2508/25ITR163-25ITR144.git